

API Documentation: Manhwa Sites Ranking

1. Overview

This API is designed to automatically discover, evaluate, and rank websites that host Manhwa (Korean webcomics). It relies on a scheduled background worker architecture to continuously discover new sites, fetch their metrics (traffic, SEO, speed, uptime), calculate a popularity score, and update their global rank.

2. Core Architecture and Data Flow

The system operates in three main phases:

Phase 1: Site Discovery

- **Component**: `DiscoverSitesJob` & `CrawlerService``
- **Action**: The system queries Google Search for terms like `"read manhwa online"` and `"manhwa sites list"` using `axios`` and `cheerio``.
- **Result**: Extracted URLs are cleaned and saved to the database using the `Site`` model. New sites are inserted dynamically without user intervention.

Phase 2: Data Collection (Scraping)

- **Component**: `ScrapeSitesJob` & `ScraperManager``
- **Action**: Iterates over all active sites (`isActive: true``). The `ScraperManager`` routes the site to a specific scraper based on the URL (e.g., `AsuraScraper`` for `asurascans.com``, `ReaperScraper`` for `reaperscans.com``, or a generic fallback).
- **Result**: Scrapers currently generate mocked performance and traffic metrics (estimated daily visits, SEO score, speed score, and uptime) and save them to the `SiteMetric`` database table. (Note: The actual web scraping logic for metrics is currently mocked with randomized data for testing purposes).

Phase 3: Scoring and Ranking

- **Component**: `RankingJob` & `RankingService``
- **Action**: Iterates over all collected `SiteMetric`` records and calculates a final score based on a weighted formula:
 - Daily Visits: 50%
 - SEO Score: 20%
 - Speed Score: 20%
 - Uptime: 10%
- **Result**: The final score is saved to the `Ranking`` model. The system then sorts all rankings in descending order by score and assigns a `rank_position`` (1st, 2nd, 3rd, etc.) to each site. (Note: There is also an auxiliary `CalculateMetricsJob`` and `PopularityService`` that calculates and saves the score directly to the `SiteMetric`` table for analytical purposes).

3. Automation and Scheduling

The system uses `adonisjs-scheduler`` to run jobs automatically in the background:

- **Daily**: The system searches Google for new manhwa websites to add to its catalog (`DiscoverSitesJob``).
- **Every 6 Hours**: The system scrapes updated metrics for all active sites (`ScrapeSitesJob``).
- **Every 6 Hours**: The system recalculates the scores and updates the leaderboard positions

(`RankingJob`).

4. API Endpoints

All endpoints are prefixed with `/api/v1/`.

Public Endpoints

- **GET /rankings**:
 - **Description**: Returns the current leaderboard of Manhwa sites.
 - **Response**: A JSON array of ranking objects ordered by `rank_position` (ascending). Each ranking object includes the preloaded `Site` data (name, url, logo).

Testing Endpoints

- **GET /test-scrape**:
 - **Description**: Manually triggers the `ScrapeSitesJob`. Forces the system to scrape metrics for all active sites immediately.
 - **Response**: `{ "status": "scraping done" }`
- **GET /test-ranking**:
 - **Description**: Manually triggers the `RankingJob`. Forces the system to recalculate all scores and assign new ranking positions immediately.
 - **Response**: `{ "status": "ranking updated" }`

User Authentication Endpoints

- **POST /auth/signup**: Register a new user account.
- **POST /auth/login**: Authenticate and receive an API access token.
- **POST /auth/logout**: Revoke the current access token (Requires Auth).
- **GET /account/profile**: Get the authenticated user's profile information (Requires Auth).

5. Database Schema

- **sites**: Stores `id`, `name`, `url`, `logo`, and `isActive` status.
- **site_metrics**: Stores historical metric snapshots including `daily_visits`, `seo_score`, `speed_score`, `uptime`, and the calculated `score`. Tied to a `site_id`.
- **rankings**: Stores the current leaderboard state including the calculated `score` and `rank_position`. Tied to a `site_id`.
- **users** & **auth_access_tokens**: Standard AdonisJS authentication tables.